PYTHON 90 DAYS CHALLENGES

Python Sets & Set Operations

Learning ? Practice ? Revision

GitHub Learning Notes

Source pages: 169?186

18 pages of original Python notes

9.4 Sets in Python

A **set** is a data structure used to store multiple **unique** values.

Syntax

```python
set_name = {value1, value2, value3}
```

Explanation

1. Sets are written using curly braces {}.
2. Set items are separated by commas.
3. Sets store only unique values.
4. Sets are unordered, so items do not have fixed positions.
5. Sets are mutable, so we can add or remove items.
6. Set elements must be immutable/hashable values.

Example 1

PYTHON CODE

```
numbers = {10, 20, 30, 40}
print(numbers)
```

{40, 10, 20, 30} : The output order may look different because sets are unordered.

Set Properties Table

Property	Meaning	Example
Unordered	Items have no fixed index	Cannot use items[0]
Unique values	Duplicate values are removed	{10, 10, 20} becomes {10, 20}
Mutable	Items can be added/removed	items.add(50)
Unindexed	No indexing/slicing	items[1] not allowed
Mixed data allowed	Can store different immutable types	{10, "Aman", True}
Fast membership check	Good for checking existence	10 in numbers

Codevi Nishchal

9.4.1 Creating Sets

Syntax

set_name = {item1, item2, item3}

1. Sets can store numbers, strings, Booleans, and tuples.
2. Sets cannot store mutable values like lists or dictionaries.
3. Empty set must be created using set(). {} creates an empty dictionary, not an empty set.

Example 1

PYTHON CODE

```
numbers = {10, 20, 30}
names = {"Aman", "Riya", "Kabir"}
mixed = {"Python", 100, 99.5, True}
empty_set = set()

print(numbers)
print(names)
print(mixed)
print(empty_set)
```

Output:
{10, 20, 30}
{'Aman', 'Kabir', 'Riya'}
{True, 99.5, 100, 'Python'}
set()
Order may differ in output.

Different Ways to Create Sets

Type	Example
Empty set	items = set()
Number set	marks = {80, 90, 75}
String set	names = {"Aman", "Riya"}
Mixed set	data = {"Aman", 20, True}
From list	items = set([10, 20, 10])
From string	letters = set("ABC")
From tuple	items = set((10, 20, 30))

CodeWithNishchal

Empty Set Important Point

```
PYTHON CODE

a = {}
b = set()
print(type(a))
print(type(b))
```

Output:
<class 'dict'>
<class 'set'>

9.4.2 Unique Values

Syntax

set_name = {value1, value2, value1}

1. Sets automatically remove duplicate values.
2. Each value appears only once.
3. This makes sets useful for removing duplicates from data.
4. Sets check uniqueness using the value, not position.

Example 1

```
PYTHON CODE
numbers = {10, 20, 10, 30, 20, 40}
print(numbers)
```

Output: {40, 10, 20, 30} => Duplicate 10 and 20 are removed.

Removing Duplicates from a List

```
PYTHON CODE
numbers = [10, 20, 10, 30, 20, 40]
unique_numbers = set(numbers)
print(unique_numbers)
```

Output: {40, 10, 20, 30}

Code With Nishchal

9.4.3 Set Elements Must Be Immutable

Syntax

set_name = {immutable_value1, immutable_value2}

Explanation

1. Set elements must be hashable.
2. Immutable values like numbers, strings, and tuples can be stored in a set.
3. Mutable values like lists, dictionaries, and sets cannot be stored in a set.
4. This is because sets internally need stable values to check uniqueness.

Example 1

```
PYTHON CODE
valid_set = {10, "Python", (1, 2)}
print(valid_set)
```

Output: {10, 'Python', (1, 2)}

Invalid example:

```
PYTHON CODE
invalid_set = {[1, 2], [3, 4]}
```

Output: TypeError: unhashable type: 'list'

Allowed vs Not Allowed Table

Value Type	Allowed in Set?	Example
Integer	Yes	{10, 20}
Float	Yes	{10.5, 20.5}
String	Yes	{"A", "B"}
Boolean	Yes	{True, False}
Tuple	Yes, if tuple contains immutable items	{(1, 2)}
List	No	{[1, 2]}
Dictionary	No	{{"a": 1}}
Set	No	{{1, 2}}

CodeWithNishchal

9.4.4 Accessing Set Values

Syntax

```
for item in set_name:
    statement
```

Explanation

1. Sets do not support indexing.
2. Sets do not support slicing.
3. Values can be accessed by looping.
4. Membership can be checked using in.

Example 1

PYTHON CODE

```
languages = {"Python", "Java", "C++"}
for language in languages:
    print(language)
```

Possible output:
Java
Python
C++
Output order may differ.

Invalid Access:

PYTHON CODE

```
languages = {"Python", "Java", "C++"}
print(languages[0])
```

Output: TypeError: 'set' object is not subscriptable

Accessing Table

Operation	Allowed?	Example
Looping	Yes	for item in items:
Membership check	Yes	"Python" in items
Indexing	No	items[0]
Slicing	No	items[1:3]

CodeWithNishchal

9.4.5 Checking Membership

Syntax

```
value in set_name
value not in set_name
```

Explanation

1. Sets are very useful for checking whether a value exists.
2. in returns True if value exists.
3. not in returns True if value does not exist.
4. Membership checking in sets is usually faster than lists for large data.

Example:

```
numbers = {10, 20, 30, 40}

print(20 in numbers)
print(50 in numbers)
print(50 not in numbers)
```

Output:
True
False
True

9.4.6 Adding Set Items

Syntax

```
Python

set_name.add(value)
set_name.update(iterable)
```

Explanation

1. add() adds one item.
2. update() adds multiple items.
3. If an added value already exists, it is not added again.
4. Sets automatically maintain uniqueness.

CodeWithNishchal

Example 1

```
PYTHON CODE
numbers = {10, 20, 30}
numbers.add(40)
numbers.update([50, 60, 20])
print(numbers)
```

Output: {40, 10, 50, 20, 60, 30}

20 was already present, so it is not duplicated.

add() vs update()

Method	Adds	Example
add()	One item	items.add(10)
update()	Multiple items	items.update([10, 20])

9.4.7 Removing Set Items

```
PYTHON CODE
set_name.remove(value)
set_name.discard(value)
set_name.pop()
set_name.clear()
```

1. remove() removes a specific item.
2. discard() also removes a specific item.
3. pop() removes a random item because sets are unordered.
4. clear() removes all items.

Example:

```
PYTHON CODE
numbers = {10, 20, 30, 40}
numbers.remove(20)
numbers.discard(50)
print(numbers)
```

Output: {40, 10, 30}

discard(50) does not give an error even though 50 is missing.

Code Nishchal

Removing Methods Table

Method	Purpose	If Value Missing
remove(value)	Removes specific value	Gives KeyError
discard(value)	Removes specific value	No error
pop()	Removes random item	Gives error if set is empty
clear()	Removes all items	No error

9.4.8 Set Operations

Set operations are used to compare or combine sets.
Main set operations:

1. Union
2. Intersection
3. Difference
4. Symmetric difference
5. Subset
6. Superset
7. Disjoint

Assume:
a = {1, 2, 3}
b = {3, 4, 5}

Set Operations Table

Operation	Symbol	Method	Meaning	Result
Union	`a	b`	a.union(b)	All unique values from both sets	
Intersection	a & b	a.intersection(b)	Common values	{3}
Difference	a - b	a.difference(b)	Values in a but not in b	{1, 2}
Symmetric difference	a ^ b	a.symmetric_difference(b)	Values not common in both	{1, 2, 4, 5}
Subset	a <= b	a.issubset(b)	Checks if all items of a are in b	False
Superset	a >= b	a.issuperset(b)	Checks if a contains all items of b	False
Disjoint	No symbol	a.isdisjoint(b)	Checks if no common items exist	False

CodeWithNishchal

Example 1

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

Output:
{1, 2, 3, 4, 5}
{3}
{1, 2}
{1, 2, 4, 5}

9.4.9 Union

Syntax

set1 | set2
set1.union(set2)

Explanation

1. Union combines two sets.
2. It returns all unique values from both sets.
3. Duplicate/common values appear only once.

Example 1

```
PYTHON CODE

a = {1, 2, 3}
b = {3, 4, 5}
result = a.union(b)
print(result)
```

Output: {1, 2, 3, 4, 5}

a = {1, 2, 3}
b = {3, 4, 5}
Union = all unique values = {1, 2, 3, 4, 5}

deWithNishchal

9.4.10 Intersection

Syntax

set1 & set2

set1.intersection(set2)

1. Intersection returns only common values.
2. Values must exist in both sets.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
result = a.intersection(b)
print(result)
```

Output: {3}

9.4.11 Difference

Syntax

set1 - set2
set1.difference(set2)

1. Difference returns values present in the first set but not in the second set.
2. a - b and b - a can give different results.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
print(a - b)
print(b - a)
```

Output:
{1, 2}
{4, 5}
a - b = values in a but not in b = {1, 2}
b - a = values in b but not in a   = {4, 5}

devWithNishchal

9.4.12 Symmetric Difference

Syntax

set1 ^ set2

set1.symmetric_difference(set2)

1. Symmetric difference returns values that are not common.
2. It removes common values from the final result.

```
PYTHON CODE
a = {1, 2, 3}
b = {3, 4, 5}
result = a.symmetric_difference(b)
print(result)
```

Output: {1, 2, 4, 5}

9.4.13 Subset, Superset, & Disjoint Sets

Syntax

set1.issubset(set2)
set1.issuperset(set2)
set1.isdisjoint(set2)

1. A subset means all values of one set exist inside another set.
2. A superset means one set contains all values of another set.
3. Disjoint sets have no common values.

```
PYTHON CODE
a = {1, 2}
b = {1, 2, 3, 4}
c = {5, 6}
print(a.issubset(b))
print(b.issuperset(a))
print(a.isdisjoint(c))
```

Output:
True
True
True

devWithNishchal

Comparison Table

Concept	Meaning	Example	Result
Subset	All items of a are in b	{1, 2} <= {1, 2, 3}	True
Superset	b contains all items of a	{1, 2, 3} >= {1, 2}	True
Disjoint	No common items	{1, 2}.isdisjoint({3, 4})	True

9.4.14 Set Methods

Method	Purpose	Example	Result / Effect
add()	Adds one item	s.add(10)	Adds 10
update()	Adds multiple items	s.update([10, 20])	Adds all items
remove()	Removes item	s.remove(10)	Error if missing
discard()	Removes item safely	s.discard(10)	No error if missing
pop()	Removes random item	s.pop()	Returns removed item
clear()	Removes all items	s.clear()	Empty set
copy()	Creates shallow copy	new = s.copy()	New set
union()	Combines sets	a.union(b)	New set
intersection()	Common items	a.intersection(b)	New set
difference()	Items in first but not second	a.difference(b)	New set
symmetric_difference()	Items not common	a.symmetric_difference(b)	New set
intersection_update()	Keeps only common items	a.intersection_update(b)	Changes a
difference_update()	Removes items found in other set	a.difference_update(b)	Changes a
symmetric_difference_update()	Keeps non-common items	a.symmetric_difference_update(b)	Changes a
issubset()	Checks subset	a.issubset(b)	True/False
issuperset()	Checks superset	a.issuperset(b)	True/False
isdisjoint()	Checks no common items	a.isdisjoint(b)	True/False

CodeWithNishchal

9.4.15 Set Operators

Set operations can also be done using operators.

Operator	Meaning	Same As
`	`	Union
&	Intersection	a.intersection(b)
-	Difference	a.difference(b)
^	Symmetric difference	a.symmetric_difference(b)
<=	Subset check	a.issubset(b)
<	Proper subset check	a < b
>=	Superset check	a.issuperset(b)
>	Proper superset check	a > b

9.4.16 Updating Sets with Operations

Syntax

set1.intersection_update(set2)
set1.difference_update(set2)
set1.symmetric_difference_update(set2)

1. Normal set operation methods return a new set.
2. Update methods change the original set.
3. These are useful when we do not need the old set.

PYTHON CODE

```
a = {1, 2, 3}
b = {3, 4, 5}
a.intersection_update(b)
print(a)
```

Output: {3}

CodeWithNishchal

9.4.17 Copying Sets

Syntax

new_set = old_set.copy()

1. copy() creates a shallow copy of a set. Direct assignment does not create a new set.
2. Direct assignment makes both variables point to the same set.

```
PYTHON CODE
a = {10, 20, 30}
b = a.copy()
b.add(40)
print(a)
print(b)
```

Output: {10, 20, 30}
{40, 10, 20, 30}

Copying Table

Code	Creates New Set?	Meaning
b = a	No	Same set reference
b = a.copy()	Yes	Shallow copy
b = set(a)	Yes	New set

9.4.18 Frozen Sets

A **frozenset** is an immutable version of a set.

Syntax

frozenset_name = frozenset(iterable)

1. A normal set is mutable. A frozenset is immutable.
2. Values cannot be added or removed from a frozenset. Frozensets can be used as dictionary keys or set elements.
3. Frozensets support read-only set operations like union, intersection, and difference.

```
PYTHON CODE
numbers = frozenset([10, 20, 30])
print(numbers)
print(type(numbers))
```

Output: frozenset({10, 20, 30})
<class 'frozenset'>

CodeWithNishchal

Invalid Operation

```
PYTHON CODE
numbers = frozenset([10, 20, 30])
numbers.add(40)
```

Output: AttributeError: 'frozenset' object has no attribute 'add'

Set vs Frozen Set Table

Point	Set	Frozenset
Mutable	Yes	No
Add items	Allowed	Not allowed
Remove items	Allowed	Not allowed
Unique values	Yes	Yes
Unordered	Yes	Yes
Can be dictionary key	No	Yes
Can be set element	No	Yes
Syntax	{1, 2, 3}	frozenset([1, 2, 3])

Frozenset Methods Table

Method	Available in Frozenset?	Reason
union()	Yes	Does not modify original
intersection()	Yes	Does not modify original
difference()	Yes	Does not modify original
symmetric_difference()	Yes	Does not modify original
issubset()	Yes	Only checks
issuperset()	Yes	Only checks
isdisjoint()	Yes	Only checks
add()	No	Modifies set
remove()	No	Modifies set
discard()	No	Modifies set
clear()	No	Modifies set
update()	No	Modifies set

CodeWithNishchal

9.4.19 Set Comprehensions

Syntax

new_set = {expression for item in sequence}

With condition:

new_set = {expression for item in sequence if condition}

1. Set comprehension is a short way to create a set.
2. It is similar to list comprehension.
3. It automatically keeps only unique values.
4. It can include conditions.

Example 1

PYTHON CODE

```
numbers = [1, 2, 2, 3, 4, 4]
squares = {number * number for number in numbers}
print(squares)
```

Output: {16, 1, 4, 9} :Duplicate input values do not create duplicate set values.

Example 2: With Condition

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = {number for number in
numbers if number % 2 == 0}
print(even_numbers)
```

Output: {2, 4, 6}

List Comprehension vs Set Comprehension

Point	List Comprehension	Set Comprehension
Brackets	[]	{}
Allows duplicates	Yes	No
Ordered	Yes	No fixed order
Result type	List	Set
Example	[x for x in data]	{x for x in data}

9.4.20 Set Conversion

Syntax

set(iterable)

list(set_name)

tuple(set_name)

1. set() converts an iterable into a set.
2. This is often used to remove duplicates.
3. A set can be converted back to a list or tuple.
4. Order may change after converting to a set.

Example 1

PYTHON CODE

```
numbers = [10, 20, 10, 30, 20]
unique_numbers = set(numbers)
print(unique_numbers)
print(list(unique_numbers))
```

Output:
{10, 20, 30}
[10, 20, 30]
Order may differ.

Conversion Table

Conversion	Code	Result Type
List to set	set([1, 2, 2])	Set
Tuple to set	set((1, 2, 2))	Set
String to set	set("banana")	Set of unique characters
Set to list	list({1, 2, 3})	List
Set to tuple	tuple({1, 2, 3})	Tuple

CodeWithNishchal

9.4.21 Set vs List vs Tuple

Point	List	Tuple	Set
Brackets	[]	()	{}
Ordered	Yes	Yes	No fixed order
Mutable	Yes	No	Yes
Allows duplicates	Yes	Yes	No
Indexing	Yes	Yes	No
Slicing	Yes	Yes	No
Best for	Changeable ordered data	Fixed ordered data	Unique unordered data
Example	[10, 20]	(10, 20)	{10, 20}

Important Set Operations Table

Operation	Code	Meaning
Create set	items = {10, 20, 30}	Creates set
Empty set	items = set()	Creates empty set
Add one item	items.add(40)	Adds one value
Add multiple items	items.update([40, 50])	Adds many values
Remove safely	items.discard(20)	Removes without error
Remove with error	items.remove(20)	Error if missing
Random remove	items.pop()	Removes random item
Empty set	items.clear()	Removes all items
Check value	10 in items	Membership check
Union	`a	b`
Intersection	a & b	Common values
Difference	a - b	Values in a, not in b
Symmetric difference	a ^ b	Non-common values
Subset	a <= b	Checks subset
Superset	a >= b	Checks superset
Copy set	items.copy()	Creates shallow copy
Frozen set	frozenset(items)	Immutable set

CodeWithNishal